HPCA 2027 Submission #NaN
Confidential Draft
Do NOT Distribute!!

QARCHGAUGE: Architecture-Guided Modeling of Practical Quantum Advantage

***Abstract*—Problem. Practical quantum advantage is often discussed as if faster quantum hardware will solve many applied problems automatically. This is not enough for computer architects. A useful claim must identify which future-system lever actually changes the outcome: logical gate latency, shot parallelism, error-correction overhead, readout/control latency, algorithmic quality, or the native HPC baseline. Without that decomposition, a speedup number does not tell architects where a quantum system should improve.**

Approach. We present QARCHGAUGE, a Perlmutter measurement and architecture-projection framework for this break-even test. QARCHGAUGE runs native baselines and quantum-circuit versions of the same workload with shared inputs and quality targets. It then converts measured circuit cost, quality gap, native runtime, circuit metadata, and repeated-execution structure into an advantage threshold over architecture variables: execution speed, shot throughput, quality recovery, and non-kernel overhead.

Result. On `Perlmutter`, our expanded `sklearn digits` sweep runs 160 cases across class pairs, sample counts, PCA dimensions, depths, and seeds. Quantum kernels require a median 421.9× speedup to match the native path; QNN/VQC requires 64.9× but usually misses native quality. We then run a 3,552-case practical suite across ML, chemistry, optimization, and simulation on up to 32 GPU nodes, plus a 104-case OpenFermion/PySCF chemistry coverage gate. Median required speedups range from 3,071.0× for simulation to 287,045.6× for optimization. The architecture implication is workload-specific: at 90% quality-gap recovery, $10^4\times$ projected execution improvement covers 54.9% of simulation cases, while $10^5\times$ covers 57.1% of chemistry cases; ML and optimization remain primarily quality-limited. These results do not claim quantum wins today. They show where future quantum architectures must spend effort before speed becomes useful.

I. INTRODUCTION

Quantum computing is often described through broad application promises: better machine learning, faster molecular modeling, improved optimization, and natural Hamiltonian simulation. For computer architects, the question is more concrete. Which future quantum-system resource should improve first: logical gate latency, two-qubit fidelity, shot throughput, readout/control latency, memory/communication near the controller, or algorithmic quality? A future quantum path is useful only if it beats a strong native HPC path on the same input, at the same quality, after all application overheads are included. Current quantum hardware is not yet large and reliable enough to answer this question directly for many practical workloads. HPC systems therefore play a second role beyond accelerating quantum-circuit simulation: they are the measurement instrument for modeling which architecture improvements would make a quantum-circuit application competitive.

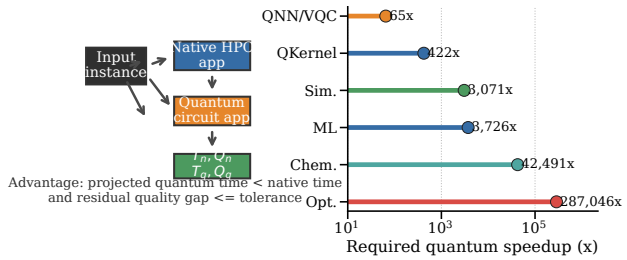


Fig. 1: Application-level comparison target. The key comparison is not simulator versus simulator. The key comparison is native application execution versus quantum-circuit application execution and projected quantum hardware execution.

Existing simulation systems provide the first part of this stack. Libraries such as `cuQuantum` execute quantum circuits on GPUs, while prior systems such as ScaleQsim and AURORA-Q improve simulation scalability on leadership-scale systems [1], [2]. These systems answer an important question: how can we simulate larger quantum circuits faster on HPC? However, simulator scalability is not the same as application-level quantum advantage. A simulator can be faster than another simulator, but the quantum-circuit application can still be slower than a native HPC application because encoding, circuit generation, repeated evaluations, measurement, optimizer steps, and postprocessing remain in the end-to-end path.

Figure 1 shows the comparison target of QARCHGAUGE. A circuit simulator is only one component of the quantum path. The full quantum path also includes application stages that do not disappear when the circuit kernel becomes faster. This creates three practical observations. **Observation 1: Simulator speed is not application speed.** Measuring only `cuQuantum` time can hide encoding, measurement, optimization, and classical control. **Observation 2: Native baselines are an architecture constraint.** Native HPC/ML applications already use mature software stacks and efficient kernels, so a fair quantum claim must compare against a native implementation of the same workload. **Observation 3: Advantage is a design-space boundary.** The goal is not only to measure today's simulator time, but to invert the measurement and compute the

TABLE I: Positioning of QARCHGAUGE relative to prior work.

Study/System	HPC sim.	Native baseline	Cost model	HW projection	Arch. bottleneck
ScaleQsim/AURORA-Q	✓				
`cuQuantum`	✓				
Benchmark suites			partial	partial	partial
Native baselines		✓			
QARCHGAUGE	✓	✓	✓	✓	✓

required quantum gate time, shot throughput, error overhead, and quality recovery for future hardware.

The same issue becomes stronger when the workload moves beyond one small ML example. Public claims about quantum computing often jump from one successful example to broad expectations such as better AI, faster drug discovery, or better optimization. A systems paper cannot validate such claims with one toy workload. It must cover representative application families and show the threshold for each family. For a practical workload, quantum advantage depends on several conditions being true at the same time: the quantum path must reach the target quality, the native baseline must be strong, the circuit must fit the available logical qubits, shots must be parallelized enough, and error overhead must not erase the speedup. QARCHGAUGE therefore reports an advantage frontier rather than a single yes/no answer.

Many previous studies focus on one part of this stack. Table I summarizes the gap. ScaleQsim and AURORA-Q improve quantum simulation on HPC systems. `cuQuantum` provides high-performance GPU primitives. Benchmark suites such as SupermarQ, QASMBench, MQT Bench, and QAOAKit improve coverage and reproducibility [3]–[6]. Native ML frameworks provide strong classical baselines. QARCHGAUGE connects these pieces into an application-level architecture requirement model: it reports whether the next useful improvement should be faster logical operations, more shot parallelism, lower error overhead, better quantum output quality, or a different algorithm.

Key Idea. QARCHGAUGE treats practical quantum advantage as an architecture design-space problem. The advantage threshold is the point where the projected quantum-circuit path becomes faster than the measured native HPC and ML path under the same workload and quality target. For each workload, QARCHGAUGE records the cost of each stage and then projects the quantum hardware region where this condition holds. The output is not one global speedup number; it is a bottleneck label that says whether future systems should prioritize gate throughput, shot parallelism, error/quality recovery, host-control overhead, or a stronger quantum algorithm.

We present QARCHGAUGE, an architecture-guided quantum-advantage modeling and analysis study for HPC systems. QARCHGAUGE starts with a controlled ML workload: `sklearn digits` native ML baselines, quantum kernel classifiers, and QNN/VQC classifiers. This first workload validates the measurement pipeline. We then scale the study to a practical suite with ML, molecular energy estimation, combinatorial optimization, and Hamiltonian simulation. The evaluation first exposes the bottleneck, then studies baseline sensitivity, scaling behavior, hardware projection, and workload-specific advantage frontiers.

This paper makes the following contributions:

- **Architecture threshold model.** We define practical quantum advantage as a same-task, same-quality break-even condition between a native HPC path and a quantum-circuit application path, then express the result as future-system requirements.
- **Logging framework.** We build a pipeline that records runtime breakdowns, quality, circuit metadata, GPU metadata, Slurm metadata, and repeat-timing evidence.
- **Practical workload evidence.** We evaluate controlled quantum ML and a suite covering ML, chemistry, optimization, and simulation, including a 3,552-case large profile, OpenFermion/PySCF chemistry, and sparse Krylov baselines.
- **Scale-out validation.** We run the large profile up to 32 GPU nodes and report weak scaling, fixed-work scaling, bundled allocation behavior, and repeat timing.
- **Architecture-region analysis.** We convert measured circuit costs into required execution speed, shot throughput, and quality-gap recovery, then classify whether each workload is speed-limited, quality-limited, shot-limited, encoding-limited, or native-dominated.

II. BACKGROUND

A. Quantum-Circuit Application Paths

Quantum-circuit applications map input data or problem instances into a sequence of quantum gates. In quantum machine learning, common paths include quantum feature maps, quantum kernels, and variational quantum circuits. A feature-map pipeline encodes classical features into circuit parameters and extracts observables or kernel values. A QNN/VQC pipeline additionally optimizes trainable circuit parameters through repeated circuit evaluations.

Repeated Execution. The key systems implication is repetition. A single circuit execution is rarely the full application. Training and evaluation require many samples, classes, optimizer steps, shots, and circuit variants. Therefore, the cost of a quantum application must include encoding, circuit construction, repeated simulation or execution, measurement, loss computation, and classical postprocessing.

Practical Application Families. QARCHGAUGE treats the first ML workload as a calibration workload, not as the whole target. Practical quantum claims usually appear in four application families. First, quantum ML claims include classification, kernel methods, and variational classifiers. Second, chemistry and drug-discovery claims include molecular energy estimation, Hamiltonian simulation, and candidate screening loops. Third, optimization claims include portfolio, routing, scheduling, and QAOA-style objective minimization. Fourth, scientific simulation claims include quantum dynamics and materials workloads. These families have different native baselines and different quantum costs. A useful advantage model must therefore report a threshold per workload family, not one global number for all quantum computing.

Terminology. The project name is QARCHGAUGE, and the paper's technical object is a *practical quantum-advantage*

threshold. This is not a claim that current quantum hardware wins on the measured applications. A threshold is the break-even condition under which the projected quantum-circuit application path becomes faster than the selected native HPC path while meeting the same quality target. We use *advantage region* for the set of speed and quality-recovery assumptions that satisfy this condition, and *architecture bottleneck* for the resource that prevents a case from entering that region. This distinction is important because a circuit simulator can be useful for measurement even when both the simulator and the projected near-term quantum path are far from application-level advantage.

B. Native Baselines and Threshold Model

Native HPC/ML baselines execute the same task directly on CPUs or GPUs. For the first QARCHGAUGE workload, the native path uses `sklearn digits` with PCA and classical classifiers such as logistic regression, MLP, and optionally SVM/RBF. These baselines are intentionally strong and simple: if a quantum-circuit model cannot beat a well-optimized native baseline at the same target quality, it does not satisfy the practical advantage threshold.

Perlmutter and cuQuantum. `Perlmutter` provides GPU nodes for quantum-circuit simulation with `cuQuantum` and cuStateVec [7]. Our environment uses an existing `cuQuantum` installation and validates it through login-node smoke tests before any allocation-consuming job. Login-node tests are used only for correctness and pipeline validation. Performance experiments run through Slurm and record queue time, elapsed time, allocation resources, and charged node-hours.

Break-even Condition. For a fixed task, dataset split, target quality, and system budget, the native application path is:

$$T_{\text{native}} = T_{\text{input}} + T_{\text{preprocess}} + T_{\text{train}} + T_{\text{eval}} + T_{\text{postprocess}}. \quad (1)$$

The quantum-circuit path simulated on `cuQuantum` is:

$$T_{\text{qc\_sim}} = T_{\text{input}} + T_{\text{encoding}} + T_{\text{circuit}} + T_{\text{cuQuantum}} + T_{\text{measure}} + T_{\text{postprocess}}. \quad (2)$$

The projected quantum hardware path is:

$$T_{\text{qhw}} = T_{\text{input}} + T_{\text{encoding}} + T_{\text{compile}} + T_{\text{execute}} + T_{\text{measure}} + T_{\text{error}} + T_{\text{postprocess}}. \quad (3)$$

The condition for projected quantum advantage is:

$$T_{\text{qhw}} < T_{\text{native}} \quad (4)$$

at the same target quality. An architecture study then asks which term must change to satisfy this inequality: native baseline strength, logical execution time, shot parallelism, error/control overhead, or quantum output quality.

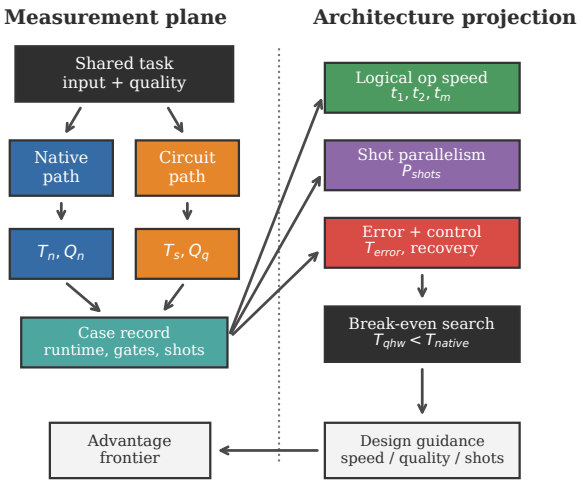


Fig. 2: Architecture and procedure of QARCHGAUGE.

III. QARCHGAUGE DESIGN

Overview of QARCHGAUGE's Design. QARCHGAUGE is designed to answer one architecture question: which future quantum-system resource must improve before a quantum-circuit application can beat the native HPC application for the same task? Figure 2 summarizes the key components. First, QARCHGAUGE uses *Shared Workload Control* to ensure that all paths use the same input instance and quality target. Second, it uses *Native Path Execution* to establish a strong classical baseline. Third, it uses *Quantum Path Execution* to run quantum kernels, QNN/VQC, VQE, QAOA, and Hamiltonian simulation circuits with `cuQuantum`. Finally, it uses *Architecture Projection* to estimate the logical-operation speed, shot throughput, quality recovery, and non-kernel overhead budgets required by future quantum systems.

Design Boundary. QARCHGAUGE is not a new quantum circuit simulator, compiler, or quantum algorithm. It treats existing simulators and native methods as execution engines behind a controlled workload interface. The design contribution is the measurement discipline and projection interface: same input, same quality metric, strongest valid native path, measured quantum-circuit path, and explicit architecture bottleneck classification. This boundary is important because faster simulation alone does not answer whether a future quantum computer would beat the native application or which architecture knob would make the difference.

A. Overall Procedure

Figure 2 shows the overall procedure. QARCHGAUGE has two main phases: *Measurement* and *Advantage Analysis*.

Measurement. When an experiment starts, QARCHGAUGE reads the workload configuration. The configuration includes the dataset, train/test split, feature dimension, model type, circuit type, shots, and target quality. QARCHGAUGE then runs the native path and the quantum-circuit path using the

same input data. This prevents unfair comparisons caused by different preprocessing or different quality targets.

Config Record. Before any path is executed, QARCHGAUGE materializes a single configuration record. This record is the logical owner of the experiment. It stores the workload family, instance ID, random seed, feature transform, native candidates, quantum circuit family, circuit depth, shot setting, and quality metric. The native and quantum paths do not choose these values independently. They receive the same record and append path-specific measurements to it. This mirrors the task-oriented style in the previous papers: the system first creates a stable representation of the work, then later components operate on that representation.

Advantage Analysis. After measurement, QARCHGAUGE reads the circuit metadata and runtime breakdown. It extracts the number of qubits, circuit depth, one-qubit gates, two-qubit gates, shots, and optimizer iterations. Using this information, QARCHGAUGE computes the quantum hardware speed required to make the projected quantum path faster than the measured native path.

Architecture Variables. The projection plane exposes four groups of future-system variables. The first group is *logical execution speed*: one-qubit gate latency, two-qubit gate latency, measurement latency, and controller scheduling overhead. The second group is *shot parallelism*: how many independent circuit evaluations can run concurrently without exceeding the device, decoder, or queue budget. The third group is *quality recovery*: algorithmic improvement, error mitigation, or fault tolerance that reduces the measured output gap. The fourth group is *host and data overhead*: encoding, compilation, optimizer feedback, and postprocessing that remain outside the quantum kernel. QARCHGAUGE keeps these variables separate so an advantage frontier can say whether an architecture needs faster logical gates, more parallel shots, lower error overhead, or a better algorithm before speed matters.

Failure Handling. QARCHGAUGE treats failed paths as measurement results rather than silently dropping them. If a native candidate fails, the record keeps the error and continues with other native candidates. If a quantum path fails, the record keeps the circuit metadata available up to the failure point and marks the case as non-advantaged. This rule is important for large sweeps because feasibility is part of the systems question. A workload that only succeeds after removing difficult cases would overstate the advantage frontier.

B. Shared Workload Control

Dataset Control. QARCHGAUGE first fixes the workload. The first full workload is `sklearn digits`. QARCHGAUGE uses a fixed train/test split and applies PCA to 4, 8, 12, and 16 dimensions. The same reduced features are used by every native and quantum model.

Instance Identity. Each measured case receives a deterministic identity. For ML, the identity includes the class pair, sample count, PCA dimension, circuit depth, and seed. For chemistry, it includes the molecule, basis, active-space setting, and circuit layer count. For optimization, it includes the graph family, graph size, seed, and QAOA layer count. For simulation, it includes the Hamiltonian model, number of qubits, evolution time, and approximation setting. This identity is used in filenames, JSON records, summaries, and paper figures. As a result, a figure bar can be traced back to the exact raw case that produced it.

Application Control. QARCHGAUGE then extends the same rule to practical workloads. For drug discovery, the native path may be a classical molecular simulation or learned surrogate, while the quantum path may be a VQE-style molecular energy circuit. For optimization, the native path may be a mixed-integer, local-search, or GPU heuristic baseline, while the quantum path may be a QAOA-style circuit. For simulation, the native path may be a classical numerical solver, while the quantum path may be a Hamiltonian simulation circuit. In every case, QARCHGAUGE keeps the task, input instance, output metric, and quality target fixed.

Quality Control. QARCHGAUGE defines the target quality before execution. The target can be a fixed accuracy threshold or a time-to-best-quality curve. This is important because a quantum path with lower accuracy cannot be claimed faster than a native path with higher accuracy.

Quality Normalization. Different application families expose different quality values, so QARCHGAUGE stores both the native metric and a tolerance-scaled gap. For higher-is-better metrics, the gap is the native score minus the quantum score. For lower-is-better metrics, it is the quantum error minus the native/reference error. The current paper uses $\epsilon_w = 0.02$ for ML/optimization accuracy or objective gaps and $\epsilon_w = 0.01$ for chemistry/simulation energy or observable gaps. These are not domain-final budgets; they are deliberately small thresholds for sensitivity analysis and are recorded in the projection artifacts. QARCHGAUGE compares the residual gap to the workload tolerance ϵ_w:

$$(1 - R)\Delta Q_w \leq \epsilon_w.$$

Here R is a hypothetical recovery factor. ΔQ_w is not a probability and is not bounded by one; values above one mean that the measured quantum path is more than one tolerance unit away from the target.

Why This Matters. Without shared workload control, the comparison becomes ambiguous. A native model may use a different feature dimension or a different split from the quantum model. QARCHGAUGE avoids this by making the workload controller the first step in the pipeline.

C. Application Path Execution

1) Native Path Execution: **Classical Baselines.** The native path executes the best available non-quantum implementation for the same input. For ML, QARCHGAUGE supports softmax regression, MLP, linear ridge classification, RBF kernel ridge classification, k-nearest neighbors, and nearest-centroid baselines without requiring an external ML package. For chemistry, the native path computes molecular energy from the same Hamiltonian by exact dense diagonalization and, in the strengthened implementation, sparse Lanczos/eigensolver

baselines for Pauli Hamiltonians. For optimization, QARCHGAUGE records exact enumeration when feasible and also greedy, local-search, and simulated-annealing baselines. For scientific simulation, the native path includes dense exact time evolution and a sparse Krylov-style evolution baseline. The native time used in the threshold is the fastest implemented model that reaches the best native quality within a small tolerance. This rule prevents a weak or low-quality native model from making the quantum path look artificially competitive.

Runtime Breakdown. QARCHGAUGE records preprocessing time, training time, inference time, and total time. It also records accuracy and model metadata. These values define T_{native}, which is the target that the projected quantum hardware path must beat.

Baseline Selection Rule. The native path may produce several valid answers for one case. QARCHGAUGE first removes candidates that do not meet the family-specific quality rule. Among the remaining candidates, it selects the fastest one as the native baseline. If no native candidate reaches the quality target, the best-quality native candidate is still recorded, but the case is marked so that the threshold analysis does not create an artificial win. This selection rule is deliberately conservative: adding a stronger native method can only make the quantum threshold harder, not easier.

2) Quantum Kernel Path: **Feature Map Execution.** The quantum kernel path maps each PCA feature vector into a quantum feature map. `cuQuantum` simulates the circuit and produces state or observable data for kernel entries. A classical classifier then consumes the kernel matrix.

Cost Breakdown. QARCHGAUGE records encoding time, circuit construction time, `cuQuantum` execution time, kernel matrix construction time, and classifier-head time. This path is useful because it separates circuit execution cost from trainable quantum-parameter optimization.

Expected Bottleneck. The kernel path can become expensive because the kernel matrix grows with the number of samples. Therefore, QARCHGAUGE must report both sample scaling and feature-dimension scaling.

Kernel Accounting Rule. The kernel path is counted at the application level, not only at the circuit-kernel level. QARCHGAUGE records how many pairwise circuit evaluations are required, how many are actually executed, whether symmetry is used, and how the resulting matrix is consumed by the classifier. This matters because a circuit that is fast for one encoded sample can still be expensive after all sample pairs are evaluated. The design therefore exposes the sample-count multiplier instead of hiding it inside one aggregate runtime.

3) QNN/VQC Path: **Parameterized Circuit Execution.** The QNN/VQC path maps the same PCA features into a parameterized quantum circuit. It then updates trainable parameters through a classical optimizer. Each optimizer step can require many circuit evaluations.

Cost Breakdown. QARCHGAUGE records ansatz type, depth, gate count, number of parameters, optimizer iterations, observable extraction time, and accuracy. This path is expected to be more expensive than the quantum kernel path because training repeatedly evaluates the circuit.

Why This Matters. QNN/VQC models are often discussed as quantum machine-learning candidates. However, their systems cost is easy to underestimate. QARCHGAUGE makes the repeated circuit evaluations visible.

Optimizer Rule. For variational paths, QARCHGAUGE records the outer-loop optimizer separately from circuit execution. The record includes the number of parameter vectors, the number of observables, the number of shots per evaluation, and the measured time spent outside `cuQuantum`. This separation is needed because future quantum hardware can reduce circuit execution time without removing the classical optimizer loop. If the optimizer or data movement dominates, the projected hardware speedup has limited effect.

D. Measurement and Threshold Analysis

JSON Record. Every run emits a JSON record. The record includes dataset metadata, model metadata, runtime breakdowns, accuracy, circuit qubits, circuit depth, gate counts, shots, GPU metadata, and Slurm metadata where available.

Summary Path. QARCHGAUGE keeps raw case records separate from derived summaries. A sweep first writes one JSON object per case or per task shard. A summarizer then validates expected case counts, merges shards, computes medians and quantiles, and writes CSV/JSON summaries used by the figures. The paper audit reads these summaries and checks that the claims in the manuscript match the stored values. This design follows the same spirit as prior systems papers: the evaluation does not depend on hand-copied numbers.

Allocation Accounting. QARCHGAUGE separates login-node smoke tests from Slurm performance runs. Login-node tests are only for correctness. Slurm runs must record elapsed time, queue time, requested walltime, allocated GPUs, and charged node-hours. This prevents short sanity checks from being mistaken for valid performance experiments.

1) Threshold Analysis: **Execution Model.** Given circuit metadata, QARCHGAUGE estimates projected quantum execution time:

$$T_{\text{execute}} = \frac{N_s(D_1 t_1 + D_2 t_2 + D_m t_m)}{P_{\text{shots}}} + T_{\text{error}}, \quad (5)$$

where N_s is the shot count, D_1 and D_2 are one- and two-qubit gate depths, D_m is measurement depth, t_1, t_2, and t_m are logical operation times, and P_{shots} is shot-level parallelism.

Projection Scope. This model is a first-order lower bound, not a validated fault-tolerant schedule. T_{error} aggregates costs not decomposed here: compilation, mapping, routing, readout, feedback, mitigation, correction, and queue/device concurrency. A fault-tolerant version can replace it with a surface-code stack using code distance, cycle time, logical-error budget, magic-state cost, and decoder latency [8], [9]. We keep the result dimensionless so stronger hardware models can be plugged in later.

Worked Hardware Instantiation. To make the abstraction concrete, the projection can be instantiated as $T_{\text{error}} =$

TABLE II: Checklist for a practical quantum-advantage claim.

Check	Question	QARCHGAUGE evidence
Same task	Are native and quantum paths solving the same input?	Shared workload config
Same quality	Does the quantum path meet the native target?	Accuracy, energy, objective, observable gap
Strong native	Is the native path the fastest valid baseline?	Baseline selection rule
End-to-end cost	Are encoding, execution, measurement, and postprocess included?	Runtime breakdown and metadata
Hardware projection	What future quantum speed is required?	Required speedup and frontier
Error/shot overhead	Could shots or error correction erase speedup?	Projection parameters and sensitivity
Architecture bottleneck	Which future-system resource matters most?	Frontier classification and taxonomy
Repeatability	Can the result be regenerated on the system?	Slurm scripts, summaries, accounting

$T_{\mathrm{compile}} + T_{\mathrm{route}} + T_{\mathrm{decode}} + T_{\mathrm{mitigate}}$ and $t_i = k_i d \tau_c$, where d is surface-code distance, τ_c is cycle time, and k_i is the number of code cycles for the logical operation. For example, an illustrative lower-bound stack with $d = 25$, $\tau_c = 1\,\mu$s, $k_1 = 1$, $k_2 = 4$, $k_m = 1$, $5\,\mu$s decoder latency per measured layer, and $P_{\mathrm{shots}} = 10^4$ parallel shots maps each measured case to a concrete logical-runtime budget. Changing any of these parameters moves the same frontier rather than changing the native baseline.

Break-even Search. QARCHGAUGE inverts the model. Instead of assuming a quantum speedup, it asks what logical gate time, shot throughput, and error overhead are needed for:

$$T_{\mathrm{qhw}} < T_{\mathrm{native}}. \tag{6}$$

This gives a hardware requirement rather than a vague claim of advantage. The requirement can be read as a design target. If a workload enters the advantage region only when P_{shots} is very large, the bottleneck is device-level or system-level parallelism. If it enters only when T_{error} is small, the bottleneck is the fault-tolerance and decoder stack. If it never enters until the quality gap closes, the bottleneck is not gate speed; it is algorithmic quality, noise resilience, or encoding.

Advantage Frontier. QARCHGAUGE does not reduce the result to one speedup number. It sweeps hardware assumptions and reports the feasible region where the projected quantum path beats the native path at the same quality. This region is the advantage frontier. If no reasonable region exists, the workload is classified by the limiting factor: quality, encoding, shots, error overhead, or native baseline strength.

Frontier Classification. After the sweep, each case is assigned a primary limiting factor. A case is speed-limited when quality is close enough but the required runtime improvement is large. It is quality-limited when even aggressive speedup does not help unless the output gap closes. It is encoding- or shot-limited when non-kernel overhead dominates the projection. It is native-dominated when a strong native baseline makes the required improvement much larger than the quantum path can plausibly recover. This classification turns the frontier into a diagnostic result instead of only a plot.

E. Advantage Claim Checklist

A practical quantum-advantage claim should pass a small set of systems checks before it is treated as meaningful. Table II lists the checks used by QARCHGAUGE. The checklist is intentionally strict because a weak native baseline, a different input instance, or a lower-quality quantum output can all create a false impression of advantage.

TABLE III: Measured workload suite.

Family	Native path	Quantum path	Quality target
ML classification	Logistic/MLP/SVM	Kernel, QNN/VQC	Accuracy, loss
Drug discovery	Classical energy/surrogate	VQE energy circuit	Energy error, rank quality
Optimization	MILP/heuristic/GPU search	QAOA circuit	Objective gap
Scientific simulation	Classical solver	Hamiltonian simulation	State/error metric

F. Workload Suite

Table III shows the measured workload suite. The digits workload is the controlled calibration case. The practical suite covers the application families that motivate many real-world quantum-computing claims.

IV. EVALUATION

This section evaluates QARCHGAUGE in the same style as our previous systems papers: first the setup, then end-to-end behavior, then component analysis, scaling, and sensitivity. The result is not a claim that quantum wins today. The result is a measured architecture model that explains how fast, how parallel, and how accurate the future quantum path must be to beat native HPC paths, and which quantum-system resource should be prioritized for each workload family.

A. Evaluation Setup

Hardware Specification. Experiments run on NERSC `Perlmutter`. Login-node smoke tests validate correctness, while performance experiments run through Slurm. GPU-node runs use A100 GPUs and record JSON outputs, Slurm metadata, elapsed time, and allocation information.

Benchmark. The controlled benchmark is the expanded `sklearn digits` sweep. It varies class pair, sample count, PCA dimension, circuit depth, and seed. The practical benchmark suite then expands the same measurement discipline to ML, chemistry, optimization, and simulation.

Baselines. Native paths are selected from workload-specific candidates. ML selects among six classifiers, optimization selects among exact and heuristic solvers, chemistry evaluates dense exact and sparse Lanczos candidates, and simulation evaluates dense exact and sparse Krylov candidates.

Feasibility. Short login runs are only correctness gates. Performance evidence must come from Slurm jobs with accounting records, raw JSON counts, and completed exit status. The completed campaign below separates calibration, application coverage, baseline stress, scaling, and timing stability.

Campaign Summary. Table IV lists the completed evidence used by the paper. The first two rows provide the main accuracy and threshold results. The next rows stress native-baseline strength, application coverage, scale-out behavior, and timing stability. This organization separates correctness gates from performance evidence and avoids treating a short smoke test as a performance claim.

Evaluation Questions. Table VI summarizes the evaluation flow. Each question maps to a measured table or figure, following the comparison–scalability–analysis structure used by our prior systems papers.

TABLE IV: Completed Perlmutter evidence campaign.

Evidence	Nodes	Cases	Role
Digits calibration	1	160	Controlled ML threshold
Practical suite	32	3,552	Main application evidence
Accept baseline gate	1	116	Chemistry/simulation baseline stress
Chemistry coverage	1	104	OpenFermion/PySCF active spaces
Weak scaling	8–32	896–3,552	Throughput scaling
Fixed-work scaling	4–32	3,552	Time-to-solution scaling
Repeat timing gate	1	12	Warmup-separated stability

TABLE V: Measured setup for the expanded digits sweep.

Item	Value
System	NERSC Perlmutter
GPU	NVIDIA A100, 1 GPU per chunk
cuQuantum	26.01.0
Dataset	`sklearn digits`
Class pairs	0-vs-1, 3-vs-8, 4-vs-9, 5-vs-8
Samples	128, 256
PCA dimensions	4, 8, 12, 16
Native models	Logistic regression, MLP
Quantum models	Quantum kernel, QNN/VQC
Cases	160

B. End-to-End Threshold Landscape

Table VII summarizes the expanded sweep. Native ML remains strong across the workload. Quantum kernels sometimes reach high quality, but the measured circuit path still requires hundreds of times faster projected execution to match the native path. QNN/VQC has a smaller required speedup in many cases, but its quality is usually below the native baseline.

Answer. The first result supports the main thesis: the useful output is not "quantum is faster" or "quantum is slower." The useful output is the hardware requirement. For this workload, even when quantum kernels achieve good accuracy, the projected execution must improve by a median 421.9× to reach native time. For QNN/VQC, the median threshold is 64.9×, but the model usually fails to match native accuracy.

Qubit Accounting. The digits circuits use one qubit per PCA feature dimension. Thus, the controlled sweep uses 4, 8, 12, and 16 qubits, matching Table V. Figures 3a and 3b are not hardware-roadmap plots over 50–1000 qubits; they are generated from the 160 measured case records and summarize threshold and quality versus the measured circuit configuration.

1) Quality Sensitivity: Table VIII shows that the threshold depends on the data pair. The easy pair, 0-vs-1, gives high quantum-kernel accuracy. Harder pairs such as 3-vs-8 and 5-vs-8 reduce quality and make a simple speedup claim invalid. This is exactly why QARCHGAUGE keeps quality and runtime together.

Answer. Quality changes the interpretation of speed. A lower runtime threshold does not imply advantage if the quantum model misses the native target quality.

2) Practical Application Landscape: The digits workload is useful because it is controlled, cheap, and easy to repeat. It is not enough for a practical quantum-advantage claim. Real users care about broader applications: AI pipelines, molecular energy estimation for drug discovery, optimization, and scientific simulation. These workloads have different native baselines and different quantum overheads.

TABLE VI: Evaluation roadmap.

Question	Goal	Main evidence
Q1	Measure end-to-end threshold landscape	Table VII, Fig. 3, Fig. 4
Q2	Explain component and baseline effects	Table IX, Fig. 5
Q3	Validate application coverage	Table X, Table XI
Q4	Separate weak and strong scaling	Fig. 6, Fig. 7, Table XII
Q5	Project advantage regions	Fig. 8, Table XIII, Fig. 9
Q6	Translate results into architecture guidance	Table XIV, Fig. 9

TABLE VII: Expanded digits sweep summary over 160 cases.

Metric	Min	Median	Max
Native accuracy	0.9219	1.0000	1.0000
Quantum kernel accuracy	0.5312	0.8750	1.0000
QNN/VQC accuracy	0.4688	0.7500	0.9531
Kernel required speedup	338.6×	421.9×	1038.7×
QNN/VQC required speedup	21.1×	64.9×	171.7×

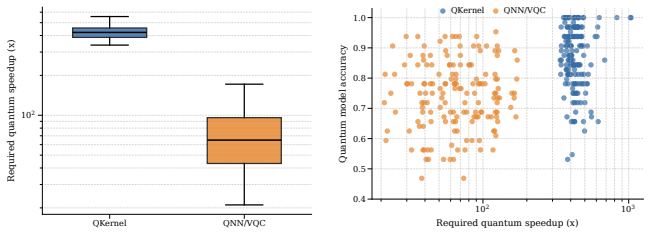
QARCHGAUGE also evaluates a practical suite beyond the controlled binary digits sweep. The suite covers ML classification, VQE-style chemistry, QAOA-style optimization, and Hamiltonian simulation. These workloads represent the application classes where users often expect quantum advantage, but the measured result shows that the threshold is application dependent and usually large. The main practical-suite result below uses the large 3,552-case profile on 32 GPU nodes. ML selects among six native classifiers, and optimization selects among exact, greedy, local-search, and annealing baselines. Chemistry and simulation use native numerical baselines on the same Hamiltonian instances and compare them with quantum-circuit executions simulated through `cuQuantum`.

Answer. The practical suite confirms that the gap is not an ML-only artifact. Every family remains outside the current advantage region, but the reason differs by family. The table shows the component mismatch directly: native paths often finish in microseconds to milliseconds, while the simulated quantum-circuit path takes about seven seconds per case. Chemistry and simulation are often closer in quality but still require large execution improvement, while ML and optimization are frequently quality-limited. The range is also large: the median threshold is 3,071.0× for simulation, 3,726.4× for ML, 42,491.4× for chemistry, and 287,045.6× for optimization.

C. Component and Baseline Analysis

Answer. The native baseline is part of the model. Strengthening the native side increases the ML threshold by 6.6× and the optimization threshold by 2.3× relative to the initial native-baseline sweep. The strong-native run completed as a one-node, four-GPU bundled allocation on `Perlmutter`. The job finished 190 cases in 6 minutes and 59 seconds with exit code 0:0, 190 raw JSON outputs, and empty stderr logs. The selected ML baselines were RBF kernel ridge, nearest centroid, kNN, softmax regression, and linear ridge depending on the case; the selected optimization baselines were mostly greedy assignment, with exact enumeration selected when it was fastest at the best cut value.

Baseline Coverage Gate. We also run a separate accept-profile gate to strengthen the two application families that are easiest to criticize as toy baselines: chemistry and scientific



(a) Threshold distribution. (b) Quality vs. threshold.

Fig. 3: Expanded digits results over 160 cases. The quantum-kernel path often reaches useful accuracy but needs hundreds of times faster execution, while QNN/VQC has a lower threshold but usually lower quality.

TABLE VIII: Class-pair sensitivity. Values are medians.

Class pair	Kernel acc.	QNN/VQC acc.
0-vs-1	0.9688	0.8750
3-vs-8	0.8125	0.7266
4-vs-9	0.9375	0.7812
5-vs-8	0.8125	0.6562

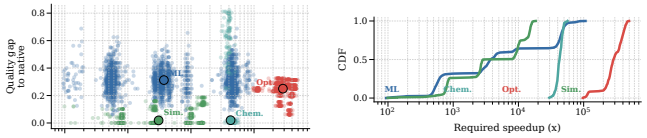
simulation. The chemistry gate includes OpenFermion and PySCF-generated H_2, LiH, and H_2O active-space Hamiltonians, in addition to the built-in minimal H_2 and molecular-chain surrogate. The committed fixture set now includes LiH and H_2O active spaces at 4, 6, and 8 qubits; the 6–8 qubit fixtures pass a login-node smoke gate before larger allocation runs. The simulation gate compares dense exact evolution with sparse Krylov evolution on TFIM and Heisenberg Hamiltonians from 4 to 8 qubits.

Answer. Table X shows that the strengthened chemistry and simulation baselines execute correctly in the same bundled one-node run. The job completed 116 cases in 6 minutes and 5 seconds with exit code 0:0; each of the four A100 tasks completed 29 cases, and stderr logs were empty. For chemistry, both dense exact and sparse Lanczos/eigensolver candidates were evaluated for every case. The selected runtime baseline is dense exact for all 56 cases because these active-space Hamiltonians are still small, but the run now uses OpenFermion and PySCF instances instead of only a hand-written surrogate. For simulation, sparse Krylov becomes the selected native method in 34 of 60 cases, especially at 6–8 qubits, showing that the native path is no longer a single dense solver.

Chemistry Active-space Coverage. We then run the expanded chemistry-only accept profile with the 4–8 qubit OpenFermion and PySCF fixture set. Job 55515248 completed 104 chemistry cases on one `Perlmutter` GPU node in 5 minutes and 23 seconds with exit code 0:0, 104 raw JSON outputs, and empty stderr. Table XI shows the larger active-space subset. Dense exact remains the selected runtime baseline because these fixtures are still small, but sparse Lanczos is the best-quality native method in 62 of 104 cases. This result upgrades the chemistry evidence from a login smoke to a completed GPU coverage gate, while still reporting the limitation that these are active-space instances rather than production drug-discovery scale.

TABLE IX: Large practical suite component summary over 3,552 cases. Values are medians.

Workload	Cases	Native ms	QSim s	Required speedup	Quality gap
ML	2,048	2.09	7.26	3,726.4×	0.3125
Chemistry	224	0.170	7.14	42,491.4×	0.0203
Optimization	768	0.024	7.02	287,045.6×	0.2500
Simulation	512	2.08	6.89	3,071.0×	0.0188



(a) Runtime-quality landscape. (b) Threshold distribution.

Fig. 4: Large practical-suite landscape over 3,552 cases. The first panel keeps both axes that matter for advantage: required speedup and quality gap. The second panel shows the full threshold distribution rather than only one bar per workload.

The last column is a tolerance-scaled error gap, not a probability or bounded accuracy loss. Values above one mean that the measured circuit path is more than one workload tolerance away from the native/reference target. This distinction is important for H_2O active-space rows: they are useful as failure evidence, but they are not near-quality advantage cases.

D. Perlmutter Scaling Analysis

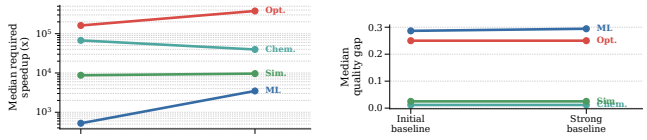
The large-profile suite contains 3,552 application cases. We evaluate two scaling modes. In weak scaling, each GPU receives roughly 28 cases, so total work grows with the number of GPUs. In fixed-work scaling, every run executes the same 3,552 cases and the chunk slots are striped across the available GPUs. Both modes use one Slurm task per A100 GPU and independent application cases; this is throughput scaling, not distributed single-circuit simulation.

1) Weak Scaling: **Answer.** All weak-scaling jobs completed with exit code 0:0. The generated raw JSON counts match the expected case counts: 896, 1,792, and 3,552. No failed case, traceback, or timeout was recorded in the task logs. The result is the desired throughput behavior for this workflow: total throughput grows with nodes while the per-GPU rate stays close to stable. This matters because the advantage model is produced by many independent application cases, so throughput scaling determines how quickly the frontier can be mapped.

2) Strong Scaling: **Answer.** The fixed-work result measures time-to-solution for the same 3,552 cases. It reduces elapsed time from 30 minutes 55 seconds on four nodes to 4 minutes 17 seconds on the matching 32-node full-profile point, a 7.2× speedup over an 8× node increase. The non-ideal 16-node point exposes a useful systems fact rather than a paper problem: application-level sweeps inherit chunk imbalance and per-case variability even when the Slurm layout is correct. This validates the bundled multi-node execution path and also motivates reporting both weak and strong scaling instead of compressing them into one bar chart.

E. Advantage Frontier and Bottleneck Analysis

The practical-suite result gives one speed threshold per case, but a speed-only threshold is still incomplete. A practical



(a) Runtime threshold shift. (b) Quality gap shift.

Fig. 5: Native baseline stress test. The slope view shows how each workload moves when the native path is strengthened, instead of hiding the shift inside grouped bars. Strengthening the native side makes the ML threshold 6.6× larger and the optimization threshold 2.3× larger than the initial native-baseline sweep.

TABLE X: Accept-profile baseline-coverage gate on one `Perlmutter` GPU node.

Workload	Cases	Required speedup	Quality gap	Selected native model
Chemistry	56	63,566.8×	0.4468	Dense exact: 56
Simulation	60	4,185.2×	0.0237	Dense exact: 26; sparse Krylov: 34

advantage claim also needs comparable output quality. We therefore convert each measured case into an advantage frontier over two axes: projected quantum execution speedup and quality-gap recovery. A point is in the advantage region only if the projected quantum path is no slower than the selected native path and the remaining quality gap is within a small workload tolerance.

Answer. The frontier separates speed-limited and quality-limited workloads. Chemistry and simulation often have small quality gaps, so their frontier is mostly pushed right by runtime. ML has a lower runtime threshold in some cases, but the quality gap must shrink before speed alone becomes meaningful. Optimization is far from the advantage region on both axes for the current QAOA configuration. This is the central point of QArchGauge: quantum advantage is a feasible region, not a single speedup slogan.

Tolerance Sensitivity. The projection table reports the tolerance used for each family. These values are intentionally strict and are treated as model inputs, not universal domain standards. A looser tolerance moves quality-limited cases toward the frontier; a stricter tolerance moves them away. The raw projection artifact records the full grid so the table can be regenerated under other tolerances without rerunning the Perlmutter jobs.

1) Hardware Projection Model: The projection model uses the measured native runtime, measured quantum-circuit simulation runtime, and measured tolerance-scaled quality gap for each case. For a projected quantum speedup S and quality-gap recovery R, a case enters the advantage region only if S is at least the measured required speedup and the residual quality gap is below the workload tolerance. Formally, a case is counted as advantaged when

$$S \geq \frac{T_{qsim}}{T_{native}} \quad \text{and} \quad (1-R)\Delta Q \leq \epsilon_w,$$

where ΔQ is the measured quality gap in the workload's natural units and ϵ_w is the workload tolerance. The recovery parameter R is not a measured error-mitigation result; it is a requirement axis. Thus, a point on the frontier says how much algorithmic, mitigation, or fault-tolerance improvement would be needed before runtime speedup matters.

TABLE XI: OpenFermion and PySCF chemistry active-space coverage gate.

Problem	Qubits	Cases	Required speedup	Tol.-scaled gap
LiH active	6	12	21,847.9×	0.3030
LiH active	8	12	821.5×	0.6299
H_2O active	6	12	21,613.0×	1.0365
H_2O active	8	12	788.0×	1.0916

TABLE XII: Large-profile weak and fixed-work scaling on `Perlmutter`.

Mode	Nodes	GPUs	Cases	Elapsed
Weak	8	32	896	4:05
Weak	16	64	1,792	4:04
Weak	32	128	3,552	4:17
Fixed work	4	16	3,552	30:55
Fixed work	8	32	3,552	15:13
Fixed work	16	64	3,552	11:05
Full profile	32	128	3,552	4:17

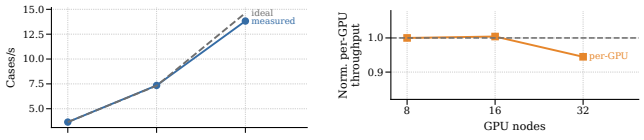
Table XIII turns the frontier into concrete hardware targets. Simulation is the closest family: with 90% quality-gap recovery, a $10^4\times$ projected execution improvement covers 54.9% of measured simulation cases. Chemistry needs a larger runtime shift: at the same recovery level, $10^5\times$ covers 57.1% of chemistry cases. ML shows the opposite behavior. A $10^5\times$ speed improvement covers only 9.8% of ML cases at 90% recovery, but almost all cases if the quality gap is fully closed. Optimization is the hardest family in the current suite; even $10^6\times$ speedup covers only 22.9% of cases at 90% recovery and requires full quality recovery to cover all cases.

Architecture Guidance. Table XIV converts the frontier into design guidance. The near-term architecture target is not universal "faster quantum." Simulation is the clearest speed/shot-parallelism target because many cases have small quality gaps and move with execution improvement. Chemistry also benefits from runtime improvement, but only after the logical stack keeps the energy gap within tolerance. ML and optimization expose a different conclusion: a faster device alone does not create advantage when the circuit output is below the native target. For those families, the architecture priority shifts toward quality-preserving encodings, error suppression, better ansatz structure, and reducing classical-quantum iteration overhead.

Answer. The hardware target is not one global number. The same projected speedup has different meaning depending on quality recovery and native-baseline strength. Faster gates help speed-limited families such as simulation and part of chemistry, but they do not rescue quality-limited ML and optimization without algorithmic improvement.

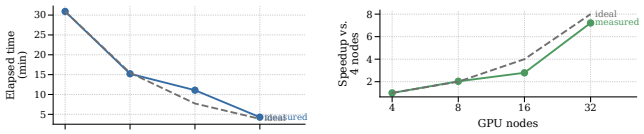
2) Bottleneck Taxonomy: The main evaluation output is not that the simulator is slow. The simulator is the measurement tool. QArchGauge uses the measurements to explain what kind of quantum hardware and algorithmic behavior would be necessary for advantage. For each workload family, it reports whether the path is limited by speed, quality, encoding, shots, error overhead, or native-baseline strength.

Figure 9 shows this classification for the 3,552-case large run using the same family tolerances as Table XIII. ML is quality-limited in all 2,048 cases because the quantum feature



(a) Total throughput. (b) Per-GPU stability.

Fig. 6: Weak scaling of the large-profile application suite. Total throughput grows nearly linearly from 8 to 32 nodes while normalized per-GPU throughput remains close to the 8-node reference.



(a) Elapsed time. (b) Speedup.

Fig. 7: Strong scaling of the fixed 3,552-case application suite. The run reduces time-to-solution from 30 minutes 55 seconds on four nodes to 4 minutes 17 seconds on 32 nodes.

path does not match the selected native classifier accuracy. Optimization is also quality-limited in all 768 cases because the current QAOA grid has a large objective gap. Chemistry has 48 speed-limited cases and 176 quality-limited cases. Simulation splits evenly, with 256 speed-limited cases and 256 quality-limited cases.

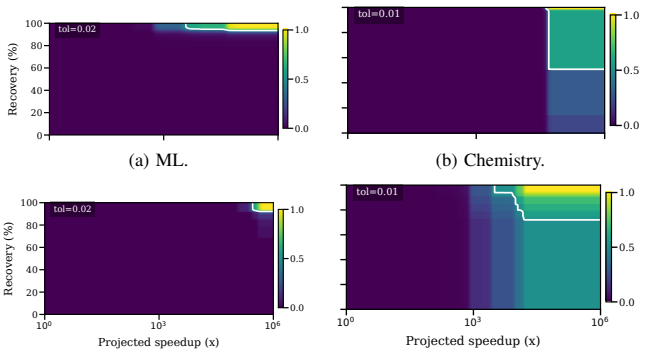
This classification is the practical insight. If a workload is speed-limited, faster logical gates or more shot parallelism may move it into the advantage region. If it is quality-limited, faster hardware alone is not enough. If it is native-dominated, the correct conclusion is that the native HPC baseline remains the better application path unless the quantum algorithm changes.

3) Sensitivity Scope: The expanded digits sweep covers class pair, sample count, PCA dimension, circuit depth, and seed. The practical suite adds workload family, native-baseline choice, chemistry grid size, circuit layer count, graph family, simulation model, and Perlmutter node count. These dimensions are sufficient for the paper's central claim: advantage thresholds are workload-specific and depend jointly on runtime and quality. The sensitivity space is still incomplete for a final hardware-roadmap study. Warmup-separated timing, repeated hardware trials, shot sensitivity, ansatz sensitivity, optimizer sensitivity, and explicit error-mitigation overhead remain outside the measured scope and are treated as threats rather than hidden assumptions.

F. Operational Stability

The official practical sweep was first run as multiple shared-GPU jobs. We then validated the bundled path with one `salloc` allocation and one multi-task `srun`. The pilot completed in 6 minutes and 54 seconds with 96 raw JSON outputs and empty stderr logs. Its systems result is operational: one `srun` step with task-local chunks selected by `SLURM_PROCID` is the reliable pattern for keeping independent cases on the allocated GPUs.

Timing Gate. Finally, a warmup-separated repeat-timing gate checks that one first-use sample does not dominate the measured path. One `Perlmutter` GPU node runs four A100 tasks; each task discards one warmup case and records three



(a) ML. (b) Chemistry.

(c) Optimization. (d) Simulation.

Fig. 8: Advantage frontier over projected quantum speedup and quality-gap recovery. Color shows the fraction of measured cases in each family that would enter the advantage region under that hypothetical hardware and algorithmic improvement. The contour marks the 50% frontier when it appears.

TABLE XIII: Projected advantage fraction from the 3,552-case strong-native suite.

Workload	Tol.	$10^4\times$, 90%	$10^5\times$, 90%	$10^6\times$, 90%	$10^5\times$, 100%	$10^6\times$, 100%
ML	0.02	6.6%	9.8%	9.8%	99.9%	100.0%
Chemistry	0.01	0.0%	57.1%	57.1%	100.0%	100.0%
Optimization	0.02	0.0%	0.0%	22.9%	0.4%	100.0%
Simulation	0.01	54.9%	71.9%	71.9%	100.0%	100.0%

measured trials. Job 55516885 completed in 58 seconds with exit code 0:0 and empty stderr. Across the 12 measured trials, the maximum quantum-runtime CV is 0.0400, so the gate supports the submitted timing evidence without replacing the large 3,552-case sweep.

V. Discussion and Threats to Validity

What the Results Prove. QArchGauge does not prove practical quantum advantage on current hardware. It proves a computer-architecture point: once the task, quality target, and native baseline are fixed, the advantage question becomes a measured design-space boundary. That boundary differs across ML, chemistry, optimization, and simulation, so one quantum-speedup claim is not enough. A useful result must say whether the next architecture dollar should buy faster logical gates, more parallel shots, lower error overhead, better controller/runtime support, or better algorithmic quality.

Native Baseline Strength. The largest threat is an underpowered native baseline. We reduce it by selecting the fastest valid method among six ML classifiers, exact/heuristic optimization, dense/sparse chemistry solvers, and dense/sparse simulation solvers. Stronger natives still exist, including CNNs or boosted trees, tuned MILP/domain heuristics, tensor networks, and CCSD(T)-class chemistry references. Adding them would usually move the frontier rightward, so our thresholds are conservative with respect to implemented baselines, not final domain limits.

Workload Representativeness. The suite covers AI, molecular energy estimation, optimization, and Hamiltonian simulation, but it is not exhaustive. It excludes cryptography, communication, sensing, and device physics because they do not fit our native-application versus quantum-circuit-application comparison. The chemistry/simulation fixtures are small active

TABLE XIV: Architecture guidance implied by the measured frontier.

Workload	Primary limiter	Future-system implication
ML	Quality gap	Gate speed has low value until encoding/noise/ansatz quality improves
Chemistry	Speed then quality	Prioritize logical-gate throughput, decoder overhead, and chemistry-quality recovery
Optimization	Quality and depth	Improve ansatz/objective quality before spending area only on faster execution
Simulation	Speed and shots	Best early target for faster logical operations and shot-level parallelism

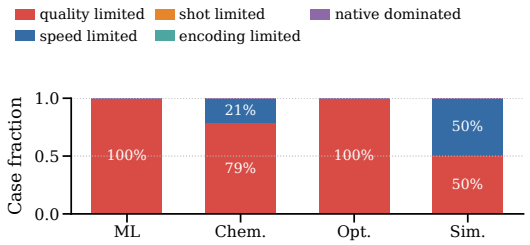


Fig. 9: Primary bottleneck taxonomy for the strong-native practical suite. ML and optimization are quality-limited in the current circuit configurations. Chemistry is mostly speed-limited, while simulation splits between speed and quality limits.

spaces, and QAOA uses fixed-grid sweeps rather than best-known tuning. The goal is break-even modeling and bottleneck exposure, not deployment-scale drug discovery or final QAOA performance.

Algorithmic Sensitivity. The current QAOA and QNN/VQC paths use fixed ansatz families and bounded optimizer grids so thousands of cases remain auditable. This can bias outcomes toward quality-limited classes. Layer-wise training, parameter transfer, natural-gradient updates, larger shot budgets, and better encodings are therefore represented by the quality-gap recovery axis, not ignored.

Simulator versus Hardware Projection. The measured quantum path uses `cuQuantum` on `Perlmutter` as instrumentation. Real hardware adds logical gate time, readout, shot limits, feedback, mapping, and error correction. The projection equation is a lower-bound model, not a calibrated timeline for any vendor. A hardware-specific study should replace T_{error} with a stack model, and fault-tolerant regimes should add code distance, cycle time, logical-error budget, magic-state overhead, and decoder latency.

Simulator Choice. The main measurements use state-vector `cuQuantum` for stable GPU instrumentation. Tensor-network, path-integral, stabilizer, or QAOA-specialized simulators could reduce $T_{\text{qc\_sim}}$ for some circuits; swapping them changes the measured quantum path but preserves the native baseline and quality test.

Name and Scope. The repository name is historical. The submitted manuscript uses the architecture-oriented display name QARCHGAUGE and frames the result as practical quantum-advantage modeling, not quantum supremacy.

Timing Stability. Small quantum-circuit experiments can be affected by Python overhead, CUDA initialization, and first-use costs. We reduce this risk with bundled Slurm runs, repeated cases, workload-level medians, login-node smoke gates, and a warmup-separated repeat gate. The gate records 12 measured trials with zero failures and maximum quantum-runtime CV 0.0400. The thresholds are therefore end-to-end application thresholds on the measured stack, not isolated kernel microbenchmarks.

Artifact and Reproducibility. The repository contains the runner, Slurm scripts, processed summaries, selected raw subsets, accounting records, and plotting inputs used by the paper. The large profile uses `QS_SWEEP_PROFILE=large`; the accept-profile gate uses `accept`. The audit script verifies the 160-case digits summary, the 3,552-case practical suite, the advantage projection, the taxonomy, the 104-case chemistry coverage gate, and the repeat-timing gate from stored JSON/CSV/accounting artifacts. The current audit status is PASS.

VI. Related Work

Computer Architecture Evaluation. Computer architecture research has long used measurement, modeling, simulation, and benchmarking to reason about systems that are too expensive or immature to build directly [10]. QARCHGAUGE follows this methodology for quantum systems: it does not claim that the simulated stack is the final hardware, but uses measured application paths to expose the architecture design space that future quantum systems must satisfy.

Quantum Simulation and NISQ Systems. Large-scale simulators such as ScaleQsim and AURORA-Q show how HPC systems extend state-vector simulation through distribution, GPUs, tiered memory, and asynchronous data movement [1], [2]. `cuQuantum` provides optimized GPU primitives [7], while NISQ studies show current-device limits [11]. QARCHGAUGE uses this substrate for a different question: what hardware and quality threshold would let a quantum-circuit application beat a native HPC application?

Benchmark Suites and Application Evaluators. SupermarQ, QASMBench, MQT Bench, QAOAKit, QED-C benchmarks, and QCHALLenge provide reusable circuits, metrics, QAOA parameters, volumetric tests, and application-aware assessment [3]–[6], [12], [13]. QARCHGAUGE is complementary: it wraps application instances with native baselines, end-to-end runtime breakdowns, and a same-quality break-even test.

Quantum Applications and Native Baselines. Quantum kernels, variational classifiers, VQE chemistry, QAOA optimization, and Hamiltonian simulation are common practical-advantage candidates [14]–[19]. Each domain also has strong native baselines and different quality metrics, making the comparison a systems problem over runtime, quality, overhead, and scale. QARCHGAUGE therefore compares quantum-circuit paths with native ML, chemistry, optimization, and simulation paths under the same workload and target quality [20].

Fault Tolerance and Hardware Realism. Fault-tolerant work and resource-estimation tools show that logical cost depends

on code distance, cycle time, layout, magic-state factories, decoder latency, and physical assumptions [9], [21], [22]. QARCHGAUGE does not validate such a stack; it reports the measured application threshold that future hardware models must satisfy.

Hybrid Orchestration and QHPC Scheduling. Hybrid systems need schedulers and workflow runtimes that coordinate queue time, fidelity, classical resources, and quantum devices [23], [24]. QARCHGAUGE does not propose a scheduler; it provides the measurement layer those schedulers need: same-task native baselines, quality targets, and threshold artifacts that say when a quantum task is worth scheduling.

VII. CONCLUSION

In this paper, we present QARCHGAUGE, a Perlmutter-based measurement and architecture-projection framework for practical quantum-advantage claims. QARCHGAUGE treats advantage as an application-level break-even condition under the same input and quality target, then maps that condition to future-system requirements. The controlled digits sweep shows median projected speedup requirements of $421.9\times$ for quantum kernels and $64.9\times$ for QNN/VQC, and the practical suite scales the analysis to 3,552 cases.

The main lesson is that no single quantum-speedup number is meaningful across applications. With 90% quality-gap recovery, a $10^4\times$ projected execution improvement covers 54.9% of simulation cases, while $10^5\times$ covers 57.1% of chemistry cases. ML and optimization also require quality recovery. Thus, the output of a practical quantum-advantage study should be an advantage frontier and architecture bottleneck taxonomy, not a yes/no slogan. For the measured suite, simulation is the clearest early target for faster logical execution and shot parallelism; ML and optimization first need quality-preserving algorithms and error behavior before hardware speed dominates.

REFERENCES

[1] C. Kim, E. Sohn, S. Kim, A. Sim, K. Wu, H. Tang, Y. Son, and S. Kim, "ScaleQsim: Highly Scalable Quantum Circuit Simulation Framework for Exascale HPC Systems," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 9, no. 3, Dec. 2025. [Online]. Available: https://doi.org/10.1145/3771577

[2] C. Kim, A. Sim, K. Wu, H. Tang, Y. Son, J. Park, and S. Kim, "AURORA-Q: Asynchronous Unified Resource Optimizer for Quantum Simulation on HPC System," Accepted to the IEEE International Conference on Distributed Computing Systems, 2026, accepted for publication.

[3] T. Tomesh, P. Gokhale, V. Omole, G. S. Ravi, K. N. Smith, J. Viszlai, X.-C. Wu, N. Hardavellas, M. R. Martonosi, and F. T. Chong, "SupermarQ: A scalable quantum benchmark suite," in *2022 IEEE International Symposium on High-Performance Computer Architecture*, 2022, pp. 587–603.

[4] A. Li, S. Stein, S. Krishnamoorthy, and J. Ang, "QASMBench: A low-level quantum benchmark suite for NISQ evaluation and simulation," *ACM Transactions on Quantum Computing*, vol. 4, no. 2, pp. 1–26, 2023.

[5] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking software and design automation tools for quantum computing," *Quantum*, vol. 7, p. 1062, 2023.

[6] R. Shaydulin, K. Marwaha, J. Wurtz, and P. C. Lotshaw, "QAOAKit: A toolkit for reproducible study, application, and verification of the QAOA," in *2021 IEEE/ACM Second International Workshop on Quantum Computing Software*, 2021, pp. 64–71.

[7] NVIDIA, "NVIDIA cuQuantum SDK Documentation," https://docs.nvidia.com/cuda/cuquantum/, 2026, accessed July 3, 2026.

[8] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, "Surface codes: Towards practical large-scale quantum computation," *Physical Review A*, vol. 86, no. 3, p. 032324, 2012.

[9] D. Litinski, "A game of surface codes: Large-scale quantum computing with lattice surgery," *Quantum*, vol. 3, p. 128, 2019.

[10] K. Skadron, M. Martonosi, D. I. August, M. D. Hill, D. J. Lilja, and V. S. Pai, "Challenges in computer architecture evaluation," *Computer*, vol. 36, no. 8, pp. 30–36, 2003.

[11] J. Preskill, "Quantum computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018.

[12] T. Lubinski, S. Johri, P. Varosy, J. Coleman, L. Zhao, J. Necaise, C. H. Baldwin, K. Mayer, and T. Proctor, "Application-oriented performance benchmarks for quantum computing," *IEEE Transactions on Quantum Engineering*, vol. 4, pp. 1–32, 2023.

[13] M. Erdmann, L. Karch, A. Awasthi, C. I. Jones, P. Bhardwaj, F. Krellner, J. Stein, C. Linnhoff-Popien, N. Kraus, J. P. Eder, S. Braun, and T. Liu, "Quantum computing—strategic recommendations for the industry," 2026.

[14] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, "Quantum machine learning," *Nature*, vol. 549, no. 7671, pp. 195–202, 2017.

[15] M. Cerezo, G. Verdon, H.-Y. Huang, L. Cincio, and P. J. Coles, "Challenges and opportunities in quantum machine learning," *Nature Computational Science*, vol. 2, no. 9, pp. 567–576, 2022.

[16] A. Kandala, A. Mezzacapo, K. Temme, M. Takita, M. Brink, J. M. Chow, and J. M. Gambetta, "Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets," *Nature*, vol. 549, no. 7671, pp. 242–246, 2017.

[17] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," 2014.

[18] I. M. Georgescu, S. Ashhab, and F. Nori, "Quantum simulation," *Reviews of Modern Physics*, vol. 86, no. 1, pp. 153–185, 2014.

[19] A. J. Daley, I. Bloch, C. Kokail, S. Flannigan, N. Pearson, M. Troyer, and P. Zoller, "Practical quantum advantage in quantum simulation," *Nature*, vol. 607, no. 7920, pp. 667–676, 2022.

[20] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[21] W. van Dam, M. Mykhailova, and M. Soeken, "Using azure quantum resource estimator for assessing performance of fault tolerant quantum computation," 2023.

[22] M. Webber, V. Elfving, S. Weidt, and W. K. Hensinger, "The impact of hardware specifications on reaching quantum advantage in the fault tolerant regime," *AVS Quantum Science*, vol. 4, no. 1, p. 013801, 2022.

[23] S. Pehlivanoglu, U. de Muelenaere, P. Kogge, and A. Sabry, "Qurator: Scheduling hybrid quantum-classical workflows across heterogeneous cloud providers," 2026.

[24] M. Tejedor, M. Grossi, C. Tüysüz, R. Rocha, and S. Vallecorsa, "Kubernetes-orchestrated hybrid quantum-classical workflows," 2026.

AI USE

OpenAI Codex assisted with drafting, editing, code generation, artifact consistency checks, and LaTeX build debugging. The authors are responsible for the final technical content, experiments, figures, citations, and claims.